Factory Auto-Scheduler
Technical Design & LLM Execution Handoff

Target stack: NestJS + PostgreSQL + Drizzle ORM
Purpose: implementation-ready specification for another LLM or engineer

Source assignment analyzed from the uploaded “Senior Backend Engineering Assessment.pdf”.

1. Assignment Decomposition

The assignment asks for a backend service in NestJS with PostgreSQL and Drizzle ORM that manages employees, car parts, and daily production requirements, then generates a schedule assigning employees to shifts while respecting skill eligibility and rest constraints. It also explicitly requires support for parts that need multiple skills, database migrations, Docker, at least one integration test, and a README that explains the design, algorithm, complexity, trade-offs, and runtime steps.

Area	What the assessor expects	Recommended interpretation
Stack	NestJS, PostgreSQL, Drizzle ORM	Do not deviate; avoid Prisma/TypeORM.
API style	REST or GraphQL	Choose REST for speed, clarity, and interview readability.
Scheduling	Functional and efficient, not mathematically perfect	Use deterministic greedy scheduling with validation.
Complex parts	Multiple skills per part	Model per-part skill requirements as many-to-many with required headcount per skill.
Production readiness	Structure, logging, edge cases, docs	Keep modular architecture, clear DTO validation, transaction boundaries, and predictable errors.

2. Delivery Strategy

- Aim for a clean, interview-grade solution, not an over-engineered optimizer.
- Implement a RESTful service with a deterministic scheduling algorithm and strong validation.
- Generate a day-level schedule for one production date at a time.
- Treat shift staffing as capacity allocation by skill, then transform that into employee assignments.
- Prioritize correctness, clarity, and explainability over “optimal” scheduling.

3. Scope Decisions and Assumptions

- One schedule request targets one production date.
- Each employee can work at most one shift per date.
- Each shift has a fixed duration of 8 hours. Use minutes for all manufacturing-time calculations.
- A production requirement says how many units of a part are needed for a given date.
- If a part requires multiple skills, each produced unit consumes labor time from each required skill lane.

- The scheduler may return unmet demand if available staff capacity is insufficient. This is better than fabricating an invalid schedule.
- The assignment example only explicitly forbids Night -> Morning back-to-back across adjacent days; for consistency, enforce a general “minimum rest window” that blocks any assignment whose previous shift end and next shift start are less than 8 hours apart.
- Do not build leave management, overtime optimization, line balancing, or skill proficiency scoring unless there is spare time.

4. Proposed Domain Model

Use a relational model that is simple, normalized, and explainable.

employees: Factory workers.

skills: Canonical skill catalog such as Welding, Painting, Electronics, Assembly.

employee_skills: Many-to-many relation between employees and skills.

parts: Car part master data, including time per unit.

part_skill_requirements: Many-to-many relation from part to required skill; includes minutes_per_unit or workers_per_unit if needed.

production_requirements: Daily demand per part and production date.

schedules: Schedule run header for a given date.

schedule_assignments: Per employee shift assignments generated by the scheduler.

5. Database Schema Blueprint

Table	Core columns	Constraints	Notes
employees	id UUID PK, name VARCHAR, is_active BOOLEAN, created_at, updated_at	name not null	Keep minimal; no unnecessary HR data.
skills	id UUID PK, code VARCHAR unique, name VARCHAR unique	unique(code), unique(name)	code useful for enums/seeding.
employee_skills	employee_id FK, skill_id FK	PK(employee_id, skill_id)	Pure join table.
parts	id UUID PK, code VARCHAR unique, name VARCHAR, default_minutes_per_unit INT	minutes > 0	default_minutes_per_unit is fallback summary value.
part_skill_requirements	id UUID PK, part_id FK, skill_id FK, minutes_per_unit INT, workers_required INT default 1	unique(part_id, skill_id), minutes_per_unit > 0	Critical table for multi-skill manufacturing.
production_requirements	id UUID PK, production_date DATE,	unique(production_date, part_id), quantity > 0	Stores daily demand input.

Table	Core columns	Constraints	Notes
	part_id FK, quantity INT		
schedules	id UUID PK, production_date DATE, status VARCHAR, unmet_demand JSONB nullable, created_at	unique(production_date)	One generated schedule per date unless versioning is added.
schedule_assignments	id UUID PK, schedule_id FK, employee_id FK, production_date DATE, shift VARCHAR, skill_id FK nullable, part_id FK nullable, assigned_minutes INT	unique(schedule_id, employee_id, shift)	Stores final roster and planned labor allocation.

6. Why this data model works

- It handles single-skill and multi-skill parts with the same model.
- It keeps scheduling logic independent from API transport concerns.
- It allows future extension to proficiency, shift preferences, overtime rules, or per-line capacity without reworking the core tables.
- It is easy to express in Drizzle and easy to explain in an interview.

7. REST API Specification

POST /skills — Create a skill.

GET /skills — List skills.

POST /employees — Create employee with skill IDs.

GET /employees — List employees with skills.

GET /employees/:id — Get employee details.

PATCH /employees/:id — Update employee name, active flag, or skill set.

POST /parts — Create part with one or more skill requirements.

GET /parts — List parts with skill requirements.

GET /parts/:id — Get part details.

PATCH /parts/:id — Update part metadata and skill requirements.

POST /production-requirements — Create or replace daily demand lines for a date.

GET /production-requirements?date=YYYY-MM-DD — Get demand for a specific date.

POST /generate-schedule — Generate schedule for a production date.

GET /schedules/:date — Get generated schedule and unmet demand for a date.

8. Request/Response Contracts

Representative payloads to keep the implementation deterministic and easy to validate.

8.1 Create employee

Request JSON:

```
{
  "name": "John Smith",
  "skillIds": ["uuid-skill-1", "uuid-skill-2"]
}
```

Response JSON:

```
{
  "id": "uuid",
  "name": "John Smith",
  "skills": [{"id": "uuid-skill-1", "name": "Welding"}]
}
```

8.2 Create part

Request JSON:

```
{
  "code": "ENGINE",
  "name": "Engine",
  "skillRequirements": [
    {"skillId": "uuid-assembly", "minutesPerUnit": 30},
    {"skillId": "uuid-electronics", "minutesPerUnit": 15}
  ]
}
```

8.3 Create production requirements

Request JSON:

```
{
  "productionDate": "2026-04-04",
  "items": [
    {"partId": "uuid-door", "quantity": 50},
    {"partId": "uuid-engine", "quantity": 20}
  ]
}
```

8.4 Generate schedule

Request JSON:

```
{
  "productionDate": "2026-04-04"
}
```

Response JSON shape:

```
{
  "scheduleId": "uuid",
  "productionDate": "2026-04-04",
  "assignments": [
    {
      "employeeId": "uuid",
      "employeeName": "John Smith",
      "shift": "MORNING",
      "allocatedSkills": [
        {"skillId": "uuid-assembly", "skillName": "Assembly", "minutes": 480}
      ]
    }
  ],
  "unmetDemand": [
    {
      "partId": "uuid-engine",
      "partName": "Engine",
      "missingUnitsEstimate": 3,
      "missingMinutesBySkill": [
        {"skillId": "uuid-electronics", "minutes": 45}
      ]
    }
  ]
}
```

9. Scheduling Model

Use a greedy, skill-capacity-based scheduler with deterministic ordering. This is the right balance for the assessment: easy to explain, easy to test, and performant enough for realistic input sizes.

1. Step 1: Load production requirements for the target date.
2. Step 2: Expand each requirement into required labor minutes per skill using part_skill_requirements.
3. Step 3: Load active employees and their skills.
4. Step 4: Load prior-day assignments to enforce rest constraints.
5. Step 5: Build candidate employee pools per shift and per skill.
6. Step 6: Allocate employees to skill buckets shift by shift until required labor minutes are satisfied or capacity is exhausted.
7. Step 7: Persist schedule header and schedule assignments inside a transaction.
8. Step 8: Return assignments plus unmet demand summary.

10. Shift and Rest Logic

Recommended fixed shifts:

- MORNING: 06:00–14:00
- SWING: 14:00–22:00
- NIGHT: 22:00–06:00 next day

Implement a generic rest-window validator rather than a hard-coded Night -> Morning check. When considering an employee for a shift, compare the end timestamp of the employee's latest previous

assignment with the start timestamp of the candidate shift. Reject the candidate if the gap is less than 8 hours. This still satisfies the assignment example and looks more production-minded.

11. Handling Parts with Multiple Skills

Do not model a multi-skill part as “choose one skill”. That would be wrong. Model it as multiple required skill contributions per unit.

- Example: one Engine may require 30 assembly minutes and 15 electronics minutes.
- If daily demand is 20 Engines, total required labor becomes 600 assembly minutes and 300 electronics minutes.
- The scheduler satisfies each skill lane independently using employees qualified for that skill.
- This is simple, relationally clean, and easy to test.

12. Deterministic Greedy Allocation Rules

- Sort production items by total required labor minutes descending so high-impact demand is handled first.
- Within each skill lane, sort candidate employees by least recent usage, then employee ID for deterministic tiebreaking.
- Assign each employee at most one shift for the target date.
- Once assigned to a shift, allocate up to full shift minutes to one or more skill demands they can satisfy. For the assessment, it is acceptable to allocate the employee to a single dominant skill for simplicity.
- If an employee has multiple skills, prefer the skill lane with the highest remaining unmet minutes.
- Track remaining minutes by skill lane and stop when all are zero or no legal employees remain.

13. Pseudocode

```
function generateSchedule(productionDate):
  requirements = fetchRequirements(productionDate)
  if requirements empty: throw 400

  skillDemands = aggregate required minutes by skill from requirements
  employees = fetchActiveEmployeesWithSkills()
  previousAssignments = fetchLatestAssignmentsBefore(productionDate)

  shifts = [MORNING, SWING, NIGHT]
  assignments = []
  assignedToday = set()

  for shift in shifts:
    for each skillDemand in skillDemands sorted by remainingMinutes desc:
      candidates = employees
        .filter(hasSkill(skillDemand.skillId))
        .filter(not assignedToday.has(employee.id))
        .filter(respectsRest(employee, shift, previousAssignments))

      for employee in candidates:
        if skillDemand.remainingMinutes <= 0: break
```

```
    allocMinutes = min(shiftCapacityMinutes, skillDemand.remainingMinutes)
    assignments.push(assign(employee, shift, skillDemand.skillId, allocMinutes))
    assignedToday.add(employee.id)
    skillDemand.remainingMinutes -= allocMinutes

unmetDemand = convertRemainingMinutesToDemandSummary(skillDemands)
saveScheduleAndAssignments(productionDate, assignments, unmetDemand)
return { assignments, unmetDemand }
```

14. Complexity

- Let R be the number of production requirement rows.
- Let K be the number of skill-demand lanes after expansion.
- Let E be the number of employees.
- Let S be the number of shifts, which is constant at 3.

A straightforward implementation that scans candidate employees for each skill lane and shift is approximately O(R + K * E * S). Since S is constant, this is effectively O(R + K * E). That is acceptable for an assessment and easy to reason about.

15. NestJS Module Structure

Module	Responsibility
database	Drizzle config, schema exports, migrations wiring.
skills	CRUD for skills.
employees	CRUD for employees and employee-skill relations.
parts	CRUD for parts and part skill requirements.
production-requirements	Daily demand intake and retrieval.
scheduler	Schedule generation service, validators, persistence, retrieval.
common	Enums, exception filters, logging, date helpers, DTO utilities.

16. Suggested Project Layout

```
src/
  app.module.ts
  main.ts
  common/
    enums/
    filters/
    interceptors/
    utils/
  database/
```

```
    db.module.ts
    db.service.ts
    schema/
      employees.ts
      skills.ts
      parts.ts
      production-requirements.ts
      schedules.ts
      index.ts
  modules/
    skills/
    employees/
    parts/
    production-requirements/
    scheduler/
test/
  scheduler.e2e-spec.ts
drizzle.config.ts
docker-compose.yml
Dockerfile
README.md
```

---

17. Implementation Plan

Phase 1 — Bootstrap

- Initialize NestJS project.
- Set up PostgreSQL connection and Drizzle configuration.
- Add Dockerfile and docker-compose.yml.
- Define enums for shifts and schedule status.

Phase 2 — Schema and migrations

- Create Drizzle schema files for all tables.
- Add primary keys, unique constraints, foreign keys, and timestamps.
- Generate and test migrations.

Phase 3 — Core CRUD APIs

- Implement skills endpoints.
- Implement employees endpoints with skill assignments.
- Implement parts endpoints with multi-skill requirements.
- Implement production requirements endpoints.

Phase 4 — Scheduling service

- Create service to aggregate labor demand.
- Create rest validator based on previous assignments.
- Implement deterministic greedy allocator.
- Persist schedule + assignments transactionally.

Phase 5 — Retrieval and error handling

- Add GET schedule endpoint.
- Add structured validation errors.
- Add graceful handling for no employees, no demand, and insufficient capacity.

Phase 6 — Tests and docs

- Write integration test proving skill and rest constraints.
- Write README with architecture, algorithm, complexity, trade-offs, and run steps.
- Seed sample data if time allows.

18. Integration Test Plan

- Test 1 — Skill constraint: create an Electronics-only part and verify a Welding-only employee is never assigned to it.
- Test 2 — Rest constraint: assign an employee to NIGHT on day D-1, generate day D schedule, and verify the same employee is not placed on MORNING.
- Test 3 — Multi-skill part: create a part requiring Assembly and Electronics, then verify the generated schedule contains labor allocation across both skill lanes or reports unmet demand accurately.
- Test 4 — Insufficient capacity: demand exceeds workforce minutes; verify schedule is still returned with unmetDemand populated instead of crashing.

19. Validation Rules

- Reject duplicate skill IDs in employee payloads.
- Reject duplicate skill requirement lines in part payloads.
- Reject production requirement payloads with duplicate part IDs for the same date.
- Reject zero or negative quantity/minutes values.
- Reject schedule generation when no production requirements exist for the requested date.
- Optionally reject re-generation for the same date unless an overwrite flag is implemented.

20. Error Handling and Logging

- Use NestJS validation pipe with class-validator DTOs.
- Return 400 for invalid input, 404 for missing resources, 409 for duplicate daily schedule generation or unique constraint violations.
- Log schedule generation start/end, input date, counts of employees/requirements/assignments, and unmet demand totals.
- Wrap schedule persistence in a transaction to avoid partial writes.

21. Trade-offs to state explicitly in README

- The scheduler is greedy and deterministic, not globally optimal.
- Employees are assigned to at most one shift per day for simplicity.
- The model uses labor minutes by skill rather than full manufacturing workflow dependencies.
- Unmet demand is returned instead of forcing invalid or hidden assumptions.
- This is the right scope for an assessment; a production version would likely add optimization heuristics, richer workforce constraints, observability, and versioned schedules.

22. What to avoid

- Do not over-engineer with event buses, CQRS, or DDD ceremony unless already fluent and very fast.
- Do not introduce a complex operations research solver; it is unnecessary risk for this assignment.
- Do not leave multi-skill parts ambiguous in the schema.
- Do not bury the core logic in controllers; keep it in services.
- Do not return a schedule without surfacing unmet demand.

23. LLM Execution Prompt

Use the following prompt verbatim or near-verbatim with the implementation LLM:

> ***Build a NestJS backend service using PostgreSQL and Drizzle ORM for the Factory Auto-Scheduler problem.***
>
> ***Mandatory requirements:***
> ***1. Use REST APIs only.***
> ***2. Implement CRUD endpoints for skills, employees, parts, and production requirements.***
> ***3. Implement POST /generate-schedule for a given productionDate.***
> ***4. Enforce:***
> ***- an employee can only be assigned where they have the required skill***
> ***- an employee cannot be scheduled in violation of an 8-hour minimum rest window; this must block Night-to-Morning across adjacent days***
> ***5. Support parts requiring multiple skills by using a many-to-many part_skill_requirements table where each row has skillId and minutesPerUnit.***
> ***6. Use a deterministic greedy scheduling algorithm based on required labor minutes by skill.***
> ***7. Persist schedules and schedule assignments in PostgreSQL.***
> ***8. Return unmet demand when capacity is insufficient.***
> ***9. Include Drizzle migrations, docker-compose.yml, README.md, and at least one integration test proving skill and rest constraints.***
>
> ***Implementation expectations:***
> ***- Keep architecture modular and interview-grade, not over-engineered.***
> ***- Use DTO validation, transaction-safe schedule persistence, and clear error handling.***
> ***- Use UUID primary keys.***
> ***- Use shift enum: MORNING, SWING, NIGHT.***
> ***- Assume 8-hour shifts and assign each employee to at most one shift per production date.***
> ***- Create clear schema definitions and relations in Drizzle.***
> ***- The README must explain database design, scheduling algorithm, complexity, trade-offs, and run instructions.***
>
> ***Preferred modules:***
> ***database, skills, employees, parts, production-requirements, scheduler, common.***

Please generate production-quality but concise code with comments only where they add value. Also generate a sample integration test and a concise README.

---

24. Final recommendation

The winning play here is disciplined simplicity. A clear REST API, a sane relational schema, and a deterministic greedy scheduler will score better than a flashy but brittle solution. The assessors explicitly signaled that maintainability, data modeling, and explainability matter as much as raw algorithmic ambition.